

# When a Machine Learns to See

Deep Learning for Images · AI and Machine Learning · Week 2, Day 2

# Where we left off

Last class we matched the data to the network. Each kind of data wants a different shape.

## Tables of numbers

Plain neural network

## Images

Today's mission



Images are the richest data and the hardest. Today we open them up.

Part 1

# Deep Learning for Images

---

# What is computer vision?

**Computer vision: building systems that look at an image and understand what is inside it.**

**1**

**Face unlock**

**2**

**Photo search**

**3**

**Medical scans**

**4**

**Self driving cars**

*In every case the input is a picture and the output is a decision.*

**Vision is already in your field?**



# Vision is already in your field

## Environmental Science

Read satellite images to track forests, floods and fires.

## Chemical Engineering

Inspect microscope images of materials and reactions.

## Architecture

Scan site photos and drawings for design and safety.

## Computer Engineering

Power cameras, face unlock and quality checks.

*Same tools, four very different jobs. Today we learn how they work.*

Part 2

# How a Computer Sees an Image

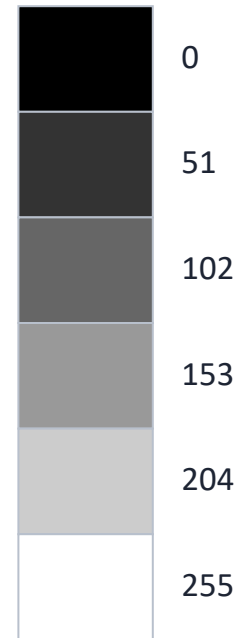
---

# An image is just a grid of numbers

|     |     |     |     |     |     |
|-----|-----|-----|-----|-----|-----|
| 20  | 40  | 60  | 90  | 120 | 160 |
| 40  | 70  | 110 | 150 | 190 | 210 |
| 70  | 120 | 170 | 200 | 230 | 245 |
| 110 | 160 | 200 | 230 | 250 | 255 |
| 150 | 200 | 235 | 250 | 255 | 255 |
| 190 | 225 | 250 | 255 | 255 | 255 |

## Brightness scale

0 is black



255 is white

**Each square is a pixel.**

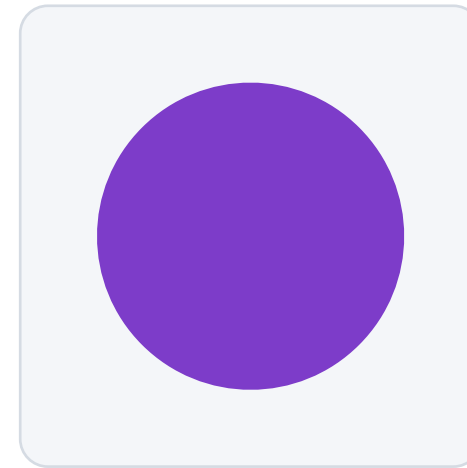
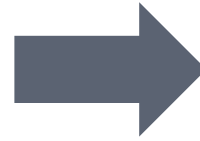
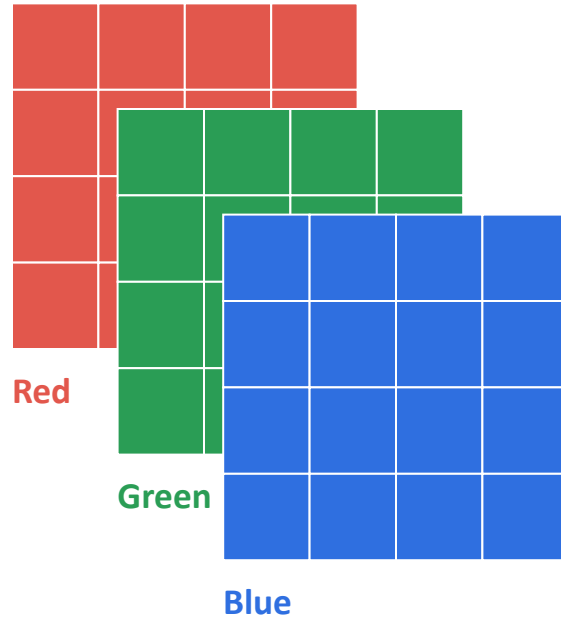
Its number says how bright it is.

A whole image is thousands of these numbers.

*This little patch is only 6 by 6. A phone photo is millions of pixels.*



# Colour adds depth: three channels



One colour pixel

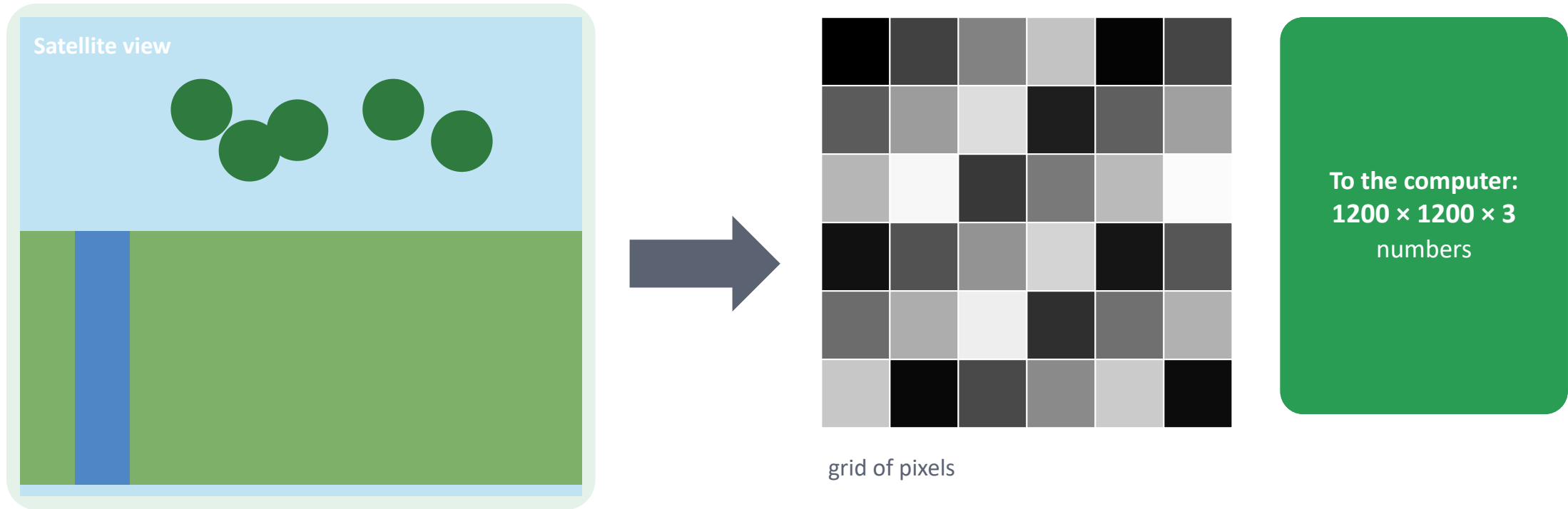
**Three numbers per pixel:**

how much red, green and blue.

Mix them and you get any colour.

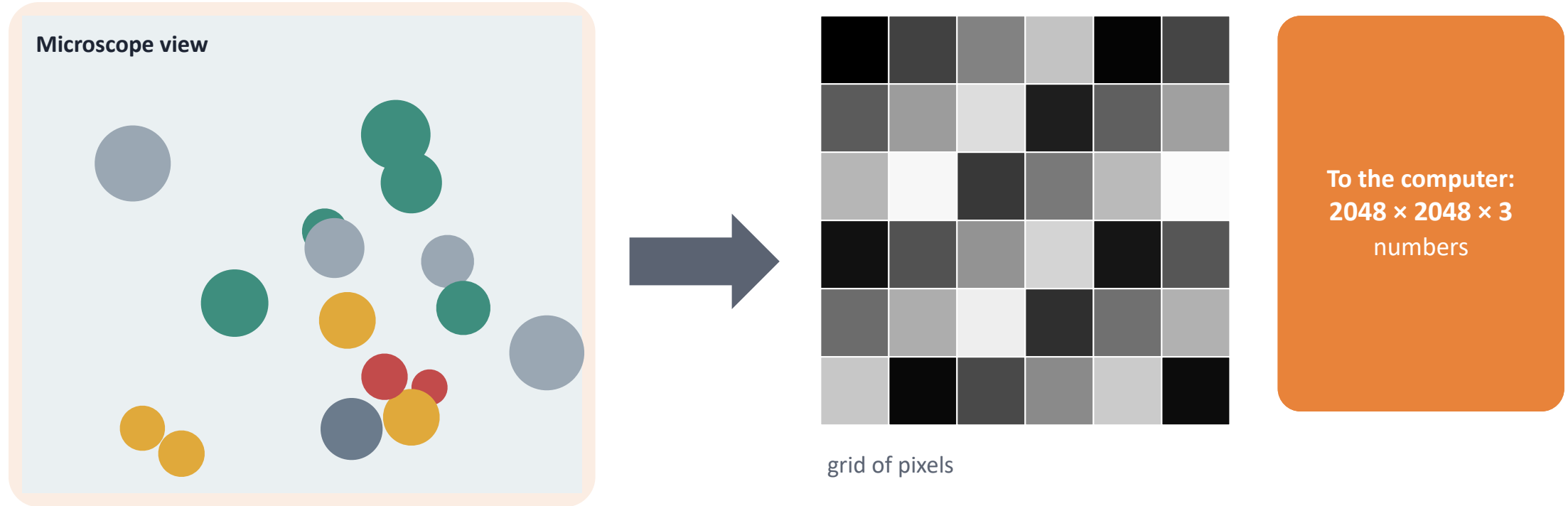
*Grayscale uses one number per pixel. Colour uses three.*

# Environmental Science: a satellite tile



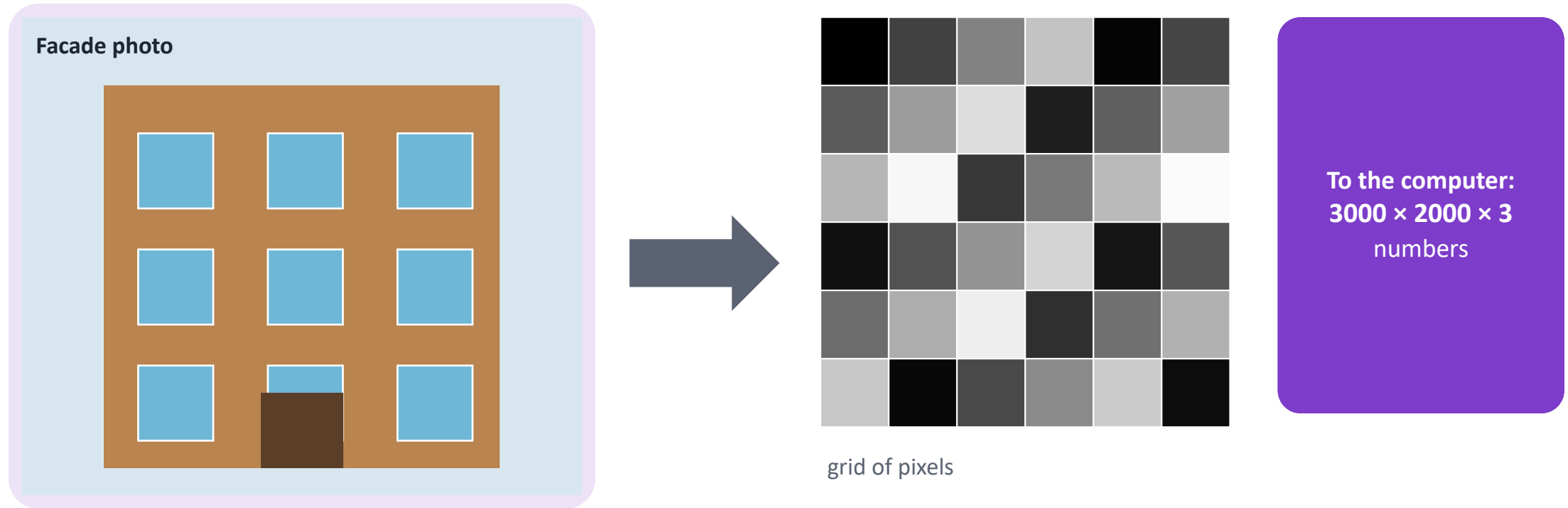
*A forest map a scientist sees becomes a giant block of numbers for the computer.*

# Chemical Engineering: a microscope image



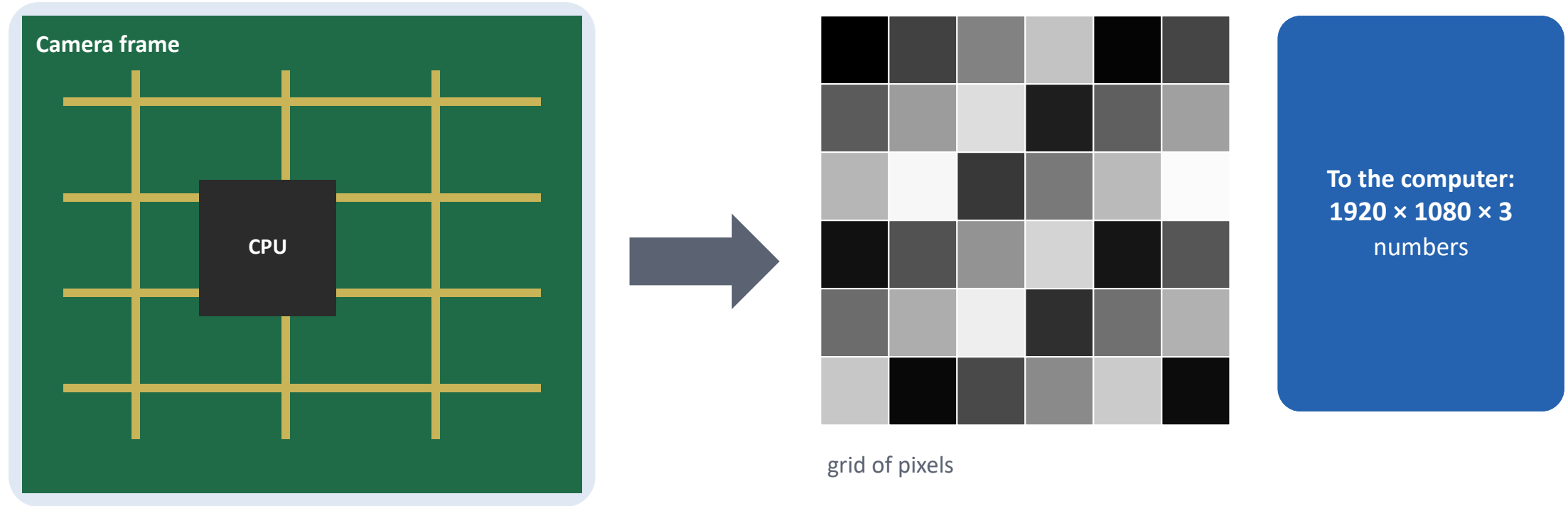
*Particles and crystals under the lens are still just pixel numbers underneath.*

# Architecture: a building facade



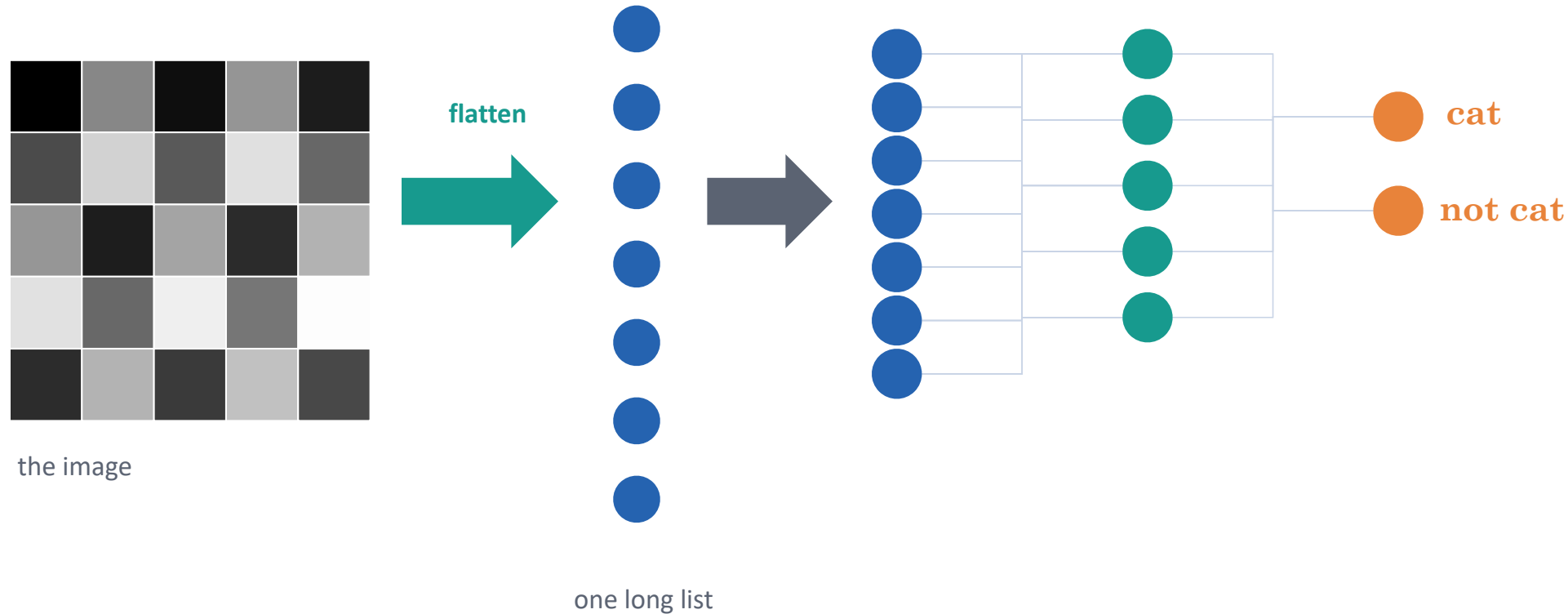
*A facade photo, windows and all, is a grid of brightness and colour values.*

# Computer Engineering: a camera frame



*Every frame your camera grabs arrives as rows and rows of numbers.*

# Feeding an image to a neural network



*We line up every pixel into one list and connect it to the network. Simple idea. Big problems ahead.*

## QUIZ 1

How does a computer store a colour photo?



As words that describe the scene



As one single number



As a grid of pixels, three numbers each



As a sound wave

Pick your answer!

## QUIZ 1

How does a computer store a colour photo?



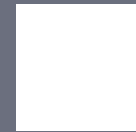
As words that describe the scene



As one single number



As a grid of pixels, three numbers each



As a sound wave

A colour image is a grid of pixels. Each pixel holds three numbers: red, green and blue.

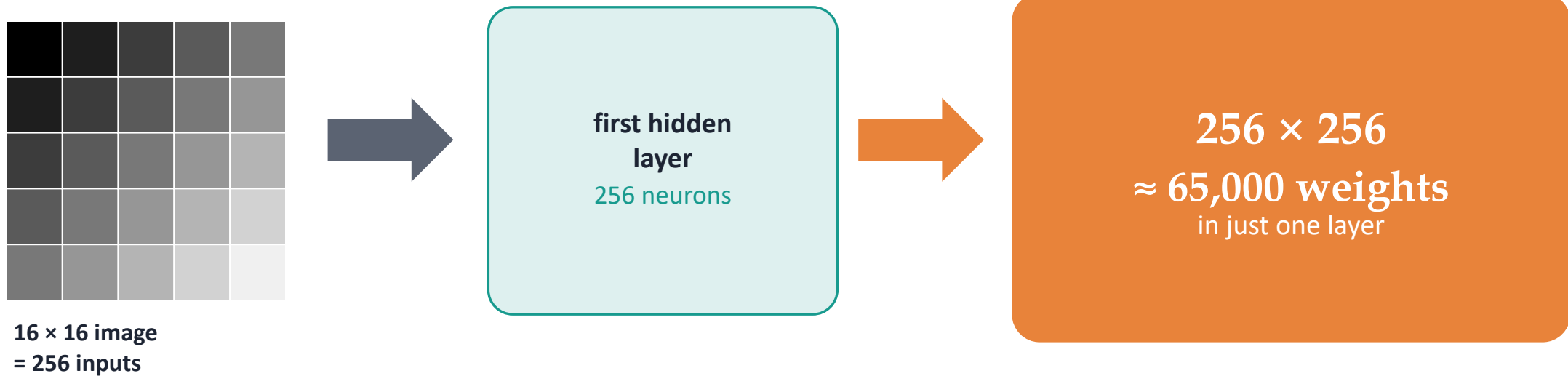


### Part 3

# Why a Plain Network Struggles with Images

---

# Wall 1: far too many connections



And that is the tiny version. Real images are far bigger.

# Now scale up to a real photo



Tiny  $16 \times 16$  image



256 inputs

# Now scale up to a real photo

Tiny  $16 \times 16$  image



256 inputs

Real  $1000 \times 1000$  colour photo



3,000,000  
inputs

That is billions of weights before training even starts. An architecture site photo is this big.

## QUIZ 2

A  $1000 \times 1000$  colour photo feeds how many input numbers into the network?



1,000



3,000



3,000,000



256

Pick your answer!

## QUIZ 2

A  $1000 \times 1000$  colour photo feeds how many input numbers into the network?



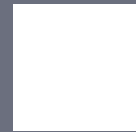
1,000



3,000



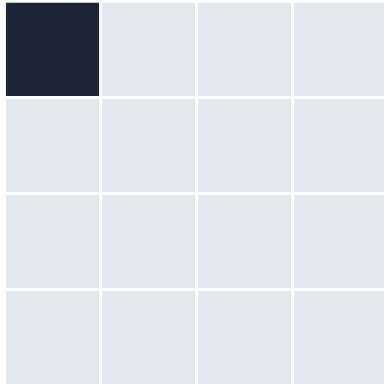
3,000,000



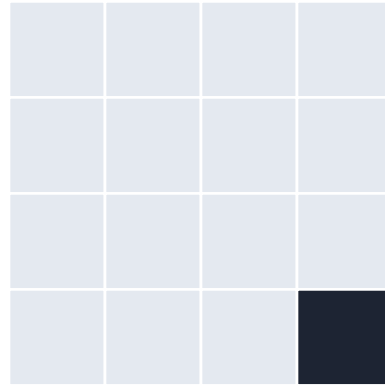
256

One million pixels times three colours equals three million numbers, before any learning begins.

# Wall 2: it cannot handle movement



same shape, top left

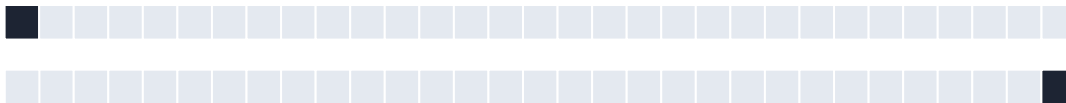


same shape, bottom right



To a plain network  
these two look completely  
**unrelated.**

The bright pixel landed in a different slot.



*Move the object and every input changes. The network has to learn it all over again.*

# Why that hurts in real work

## Architecture

A crack can appear anywhere on a wall.

## Computer Engineering

A defect can sit in any corner of a chip.

## Environmental Science

A wildfire can start in any tile of the map.

**A plain network must relearn the very same pattern in every position.**



### QUIZ 3

Move the same object to a new corner of the photo. A plain network then...



Sees almost the same thing



Sees a totally new input and gets confused



Ignores the move



Runs faster

Pick your answer!

## QUIZ 3

Move the same object to a new corner of the photo. A plain network then...



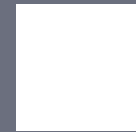
Sees almost the same thing



Sees a totally new input and gets confused



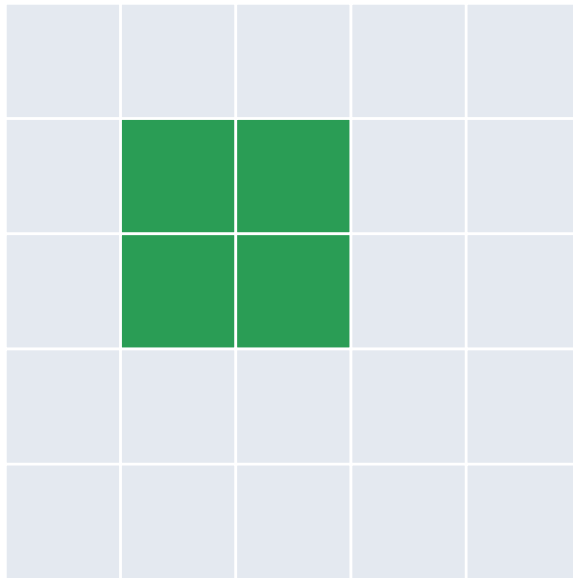
Ignores the move



Runs faster

Flattening ties each pixel to a fixed slot, so a shifted object lands on new inputs and looks brand new.

## Wall 3: it throws away the picture



after flattening, those neighbours are torn apart

Nearby pixels build edges and textures. Flattening forgets who sat next to whom.

# Three walls

1

## Too many connections

Millions of weights to train.

2

## Blind to position

A shift looks brand new.

3

## Loses the layout

Forgets which pixels were neighbours.

A plain network was never built for pictures. We need a smarter design.

## QUIZ 4

Which of these is NOT a problem when you push an image through a plain network?



Too many weights to train



It forgets which pixels were neighbours



A shifted object looks brand new



It cannot read numbers at all

Pick your answer!

## QUIZ 4

Which of these is NOT a problem when you push an image through a plain network?



Too many weights to train



It forgets which pixels were neighbours



A shifted object looks brand new



It cannot read numbers at all



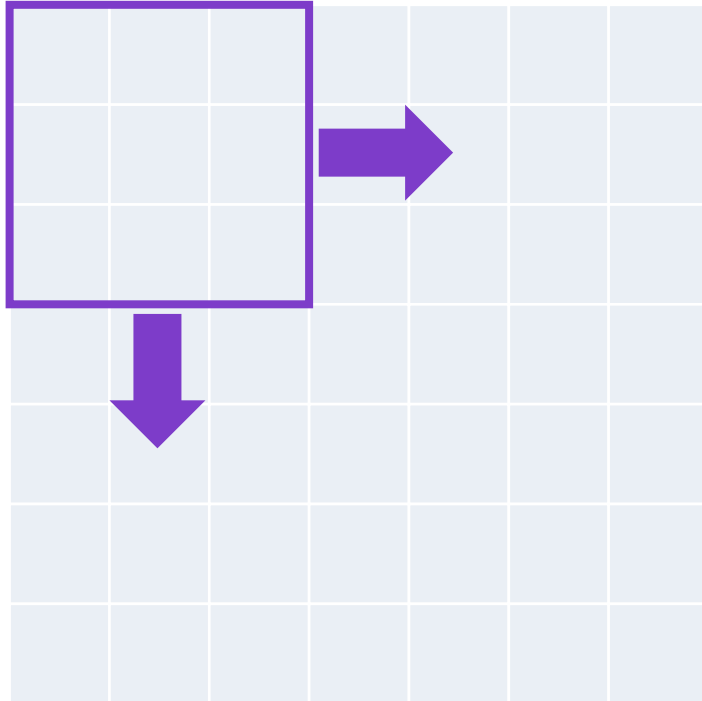
Plain networks read numbers fine. The real walls are size, position blindness and lost layout.

Part 4

# The Right Tool: Convolutional Neural Networks

---

# The big idea: slide a tiny window



**Look at small patches,  
one at a time,  
instead of the whole image at once.**

*The window slides across the image, left to right and top to bottom.*



# A filter looks for one feature

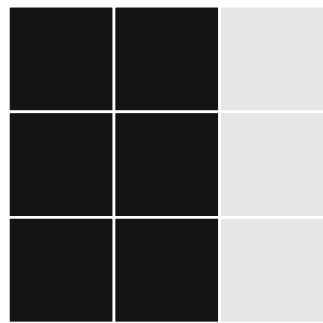
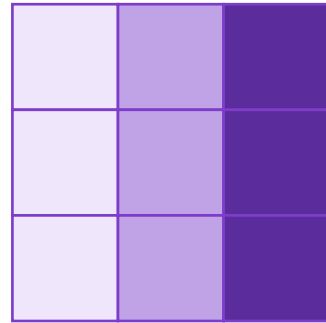


image patch

×



edge filter

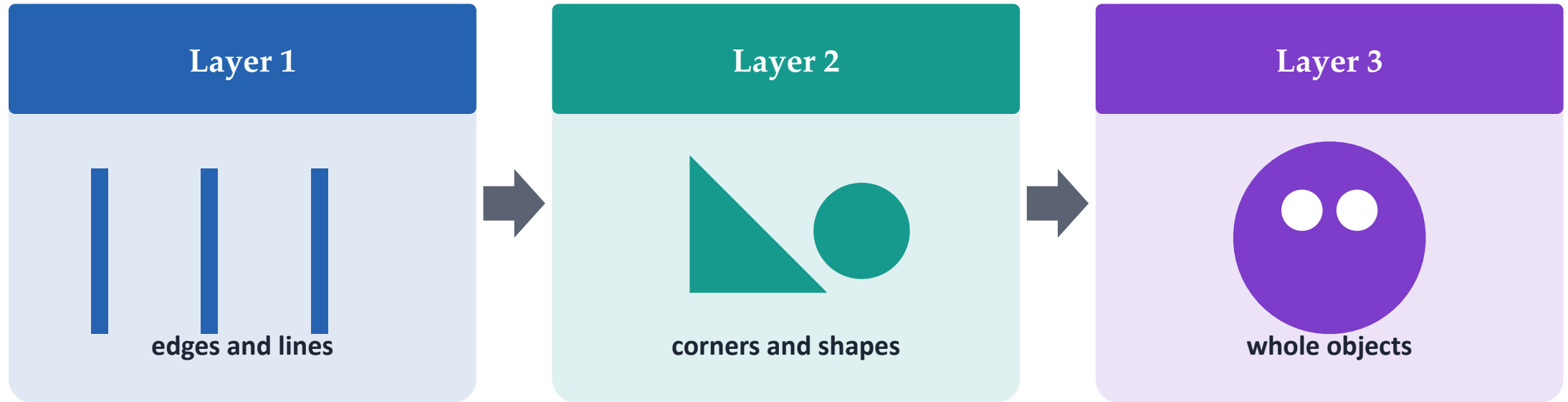


strong match

**Multiply and add inside  
the little window.**  
A high score means the  
feature is there.

*This window is called a filter or kernel. No big formulas needed.*

# The network learns its own filters

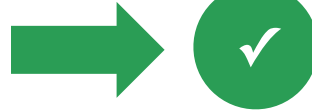


*Early layers find simple edges. Deeper layers stack them into shapes and then objects.*



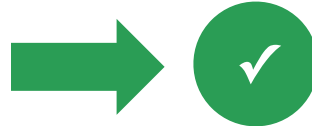
# One idea, three walls gone

**Too many weights**



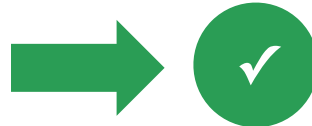
One small filter is reused everywhere, so very few weights.

**Blind to position**



The same filter slides over every spot, so position no longer matters.

**Loses the layout**



It works on 2D patches, so the picture structure stays intact.

**This is the right tool for images.**

## QUIZ 5

What is the core trick of a convolutional neural network?



It memorises every photo



It slides small shared filters across the image



It deletes the colour



It makes the image bigger

Pick your answer!

## QUIZ 5

What is the core trick of a convolutional neural network?



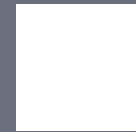
It memorises every photo



It slides small shared filters across the image



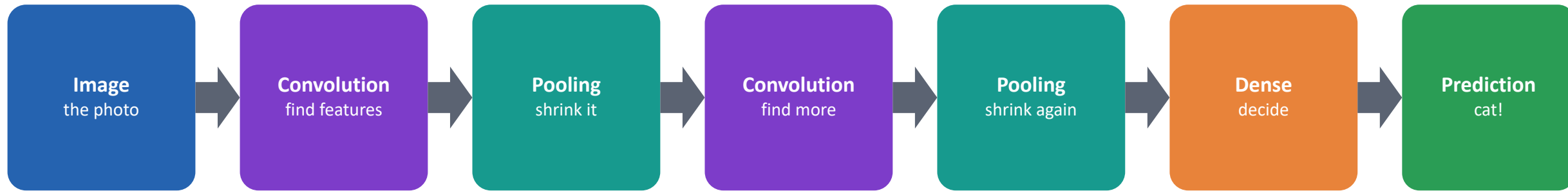
It deletes the colour



It makes the image bigger

A CNN reuses small filters across the whole image. Few weights, handles movement, keeps the 2D layout.

# Anatomy of a CNN



Convolution layers find features. Pooling layers shrink the picture to keep only what matters.

Repeat a few times, then a small dense layer makes the final call. Same neuron idea, arranged for images.

# CNNs at work in your field

## Environmental Science

Spot deforestation and wildfires in satellite tiles.

## Chemical Engineering

Count particles and flag defects in microscope images.

## Architecture

Classify building styles and detect cracks and damage.

## Computer Engineering

Face unlock, object detection and quality checks.

*Same CNN engine, tuned to each field with its own images.*

## QUIZ 6

A satellite keeps photographing a forest. Which model best spots new cleared patches anywhere?



A spreadsheet



A plain fully connected network



A convolutional neural network



None, it is impossible

Pick your answer!



## QUIZ 6

**A satellite keeps photographing a forest. Which model best spots new cleared patches anywhere?**



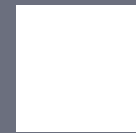
A spreadsheet



A plain fully connected network



A convolutional neural network



None, it is impossible

A CNN scans the whole image with shared filters, so it catches a pattern wherever it appears.

Bonus

# One Last Concept for Today: Transfer Learning

---

# Training a CNN from scratch is expensive

1

**Millions of images**

to learn good filters

2

**Many GPUs**

heavy computing power

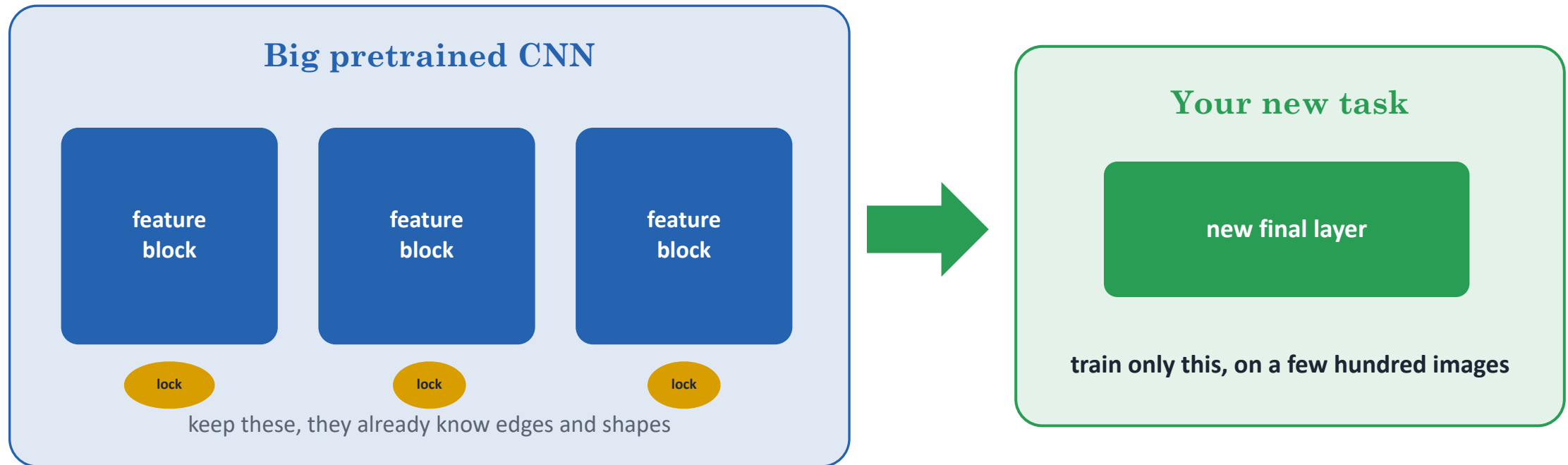
3

**Days of training**

slow and costly

**Your lab will not have millions of labelled images. So, we borrow instead.**

# Borrow a network that already learned to see

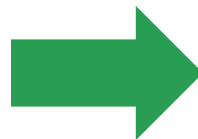


*Reuse the part that learned to see. Replace only the last layer for your own labels.*

# Like hiring an expert



An expert who already  
**knows how to see**  
edges, shapes, textures



You only teach them  
**your own labels**  
a quick lesson, not years of school

*You skip the years of learning to see and jump straight to your problem.*

# Why this is perfect for you



**Small dataset is enough**

A few hundred images can work.



**Trains in minutes**

Not days. Often on a laptop.



**Easy to share**

Many pretrained models are free.

In your field: an architecture student with 300 facade photos, a few hundred labelled satellite tiles, or a small set of microscope or chip images. Transfer learning makes all of these possible.

## QUIZ 7

Why is transfer learning so useful for a student project?



It needs millions of images



It reuses a model that already learned features



It only works on text



It removes the need for any data

Pick your answer!

## QUIZ 7

Why is transfer learning so useful for a student project?



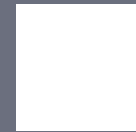
It needs millions of images



It reuses a model that already learned features



It only works on text

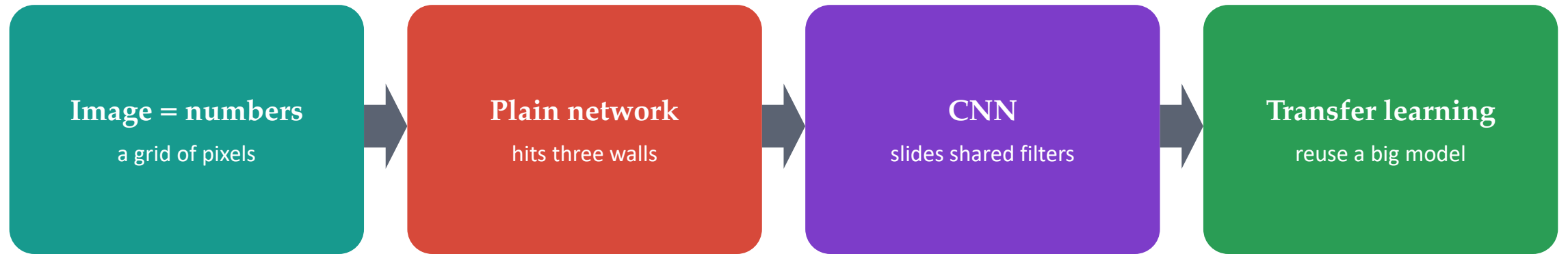


It removes the need for any data

A pretrained CNN already knows edges, shapes and textures, so a few hundred of your own images is enough.



# Putting it all together



*From raw pixels to a smart, reusable image model. That is deep learning for images.*

# Key takeaways

- **An image is just a grid of numbers.**  
Grayscale uses one per pixel, colour uses three.
- **A plain network hits three walls on images.**  
Too many weights, blind to position, loses the layout.
- **CNNs slide small shared filters.**  
Few weights, position aware, structure kept. The right tool.
- **Transfer learning reuses big models.**  
Small dataset, fast training, works in your field.

# Questions?

---

Next time: teaching machines to work on real applications in computer vision